26. Hashing and Bloom Filters (Infra-Relevant)

26. Hashing and Bloom Filters (Infra-Relevant)

Phase: Phase 4 — Algorithms and data structures
Estimated time: 1–2 days
Slug: `26-hashing-bloom`

Learning objectives

- Explain hash maps vs cryptographic hashes
- Implement a simple bloom filter
- Reason about false positives
- Apply hashing to caches and shards

Prerequisites

Complete prior numbered lessons. Type every example yourself.

Hash tables (maps)

Go's `map` is a hash table: average O(1) lookup. Hash collisions are handled internally.

Cryptographic vs non-crypto

- `hash/fnv`, `maphash`: fast, non-crypto
- `crypto/sha256`: integrity / security

Do not use non-crypto hashes for security tokens.

Bloom filters

Probabilistic set: can say "possibly in set" or "definitely not". Uses bit array + k hash functions. False positives; no false negatives (for standard bloom).

Used in: databases (skip disk reads), caches, distributed systems membership tests.

Tradeoff: memory vs false positive rate.

Common mistakes (especially from Python instincts)

- Using MD5 for security (broken)
- Ignoring bloom false positives in logic
- Rolling your own crypto

Exercises

Work in `exercises/26-hashing-bloom/`. Read that folder's `README.md`.

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/26-hashing-bloom/` until you have an honest failing attempt.

Self-check

1. What can a bloom filter never say?
2. When use sha256 vs fnv?
3. Why maps are O(1) average?

Further reading

- https://pkg.go.dev/hash
- https://en.wikipedia.org/wiki/Bloom_filter